ACKNOWLEDGEMENT

First and foremost, I want to thank everyone that has helped me during this Internship I period. As this would not be possible without them.

I would like to express my deepest gratitude for the internship opportunity granted by **H.E. Mr. Heang Sotheayuth**, Deputy Secretary General of The Council for the Development of Cambodia (CDC). and to the president, **H.E. Dr. Seng Sopheap**, and vice-president, **H.E. Hean Samboeun**, of Cambodia Academy of Digital Technology (CADT) for making it possible for us to participate and study for our bachelor's degree program at CADT.

I want to thank my advisor, ___, for his valuable guidance and advice. Which has helped me a lot in understanding this Internship II program.

I am also very grateful to my supervisor and the project manager, **Mr. Khouch Koeun**, the technical lead **Mr. Van Dara**, and everyone else at the Council for the Development of Cambodia for being very supportive and helpful throughout the internship process, they have shared valuable insights to me whether it is technical or knowledge about the working environment.

I also would like to thank the Department of Computer Science and Career Center at Cambodia Academy of Digital Technology (CADT) for organizing this internship I program and making sure everything goes smoothly.

Finally, I would like to thank my friends and family for their support, especially the friends who also happen to take this internship program at CDC.

មូលន័យសង្ខេប

ក្នុងគម្រោងនេះ ខ្ញុំបានអភិវឌ្ឍ...

ABSTRACT

…

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

Abbreviation	Explanation
AI	Artificial Intelligence
API	Application Programming Interface
CADT	Cambodia Academy of Digital Technology
CDC	Council for the Development of Cambodia
CDN	Content Delivery Network
CI	Continuous Integration
CIB	Cambodia Investment Board
CRDB	Cambodian Rehabilitation and Development Board
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSEZB	Cambodia Special Economic Zone Board
E2E	End-to-End
FCM	Firebase Cloud Messaging
G2B	Government to Business
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IDT	Institute of Digital Technology
IMAP	Internet Message Access Protocol
ITPR	Information Technology and Public Relation Directorate
JSON	JavaScript Object Notation
JWT	JSON Web Tokens
MFA	Multi-Factor Authentication

Abbreviation	Explanation
OCR	Optical Character Recognition
ODA	Official Development Assistance
QIP	Qualified Investment Project
REST	Representational State Transfer
SDK	Software Development Kit
SEZ	Special Economic Zone
SMTP	Simple Mail Transfer Protocol
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
ZTA	Zero Trust Architecture

I. INTRODUCTION

The internship program is a cornerstone of the bachelor's curriculum at the Institute of Digital Technology (IDT), Cambodia Academy of Digital Technology (CADT). Designed to bridge the gap between academic learning and professional practice, the program places students within real organizations where they can contribute to and learn from active projects in their field of study. This report documents the work undertaken, the challenges encountered, and the knowledge gained throughout the entirety of this internship period.

1.1. Presentation of the Internship

I conducted my internship at the Council for the Development of Cambodia (CDC), embedded within the Information Technology and Public Relation Directorate (ITPR). Over the course of the program, I was entrusted with the end-to-end development of a specialized mobile email client engineered for government officials responsible for managing the CDC's investor relations. This involved full-stack responsibilities spanning mobile application development, backend API design, and integration with existing internal microservices.

1.1.1. Objective of Internship

The primary objective of this internship, as defined by the IDT program guidelines, is to provide students with a structured opportunity to apply the theoretical knowledge and technical skills developed during their academic studies to the demands of a real professional environment. The program is intended to expose students to industry workflows, workplace communication norms, and the practical constraints that shape software development in an institutional context.

Beyond technical growth, the internship serves as a platform for students to evaluate their own professional strengths and identify areas for continued development. For the host organization, it provides the opportunity to assess a student's technical competency, initiative, and collaborative ability, a mutual exchange that may lead to future employment or continued collaboration.

1.1.2. Duration of Internship

The internship commenced on January 5th, 2026, and concluded on July 5th, 2026, spanning a total of six months (24 weeks). In accordance with the CADT Year 4 curriculum requirements, a minimum of 960 hours of documented professional work was completed over this period.

1.2. Presentation of the Organization

This section provides an overview of the Council for the Development of Cambodia, the organization at which the internship was conducted. It covers the CDC's mandate, the services it provides, its institutional vision and mission, and its internal structure. Understanding the organizational context is essential to appreciating why this project was prioritized and how it supports the CDC's broader operational goals.

1.2.1. General Information of the Organization

Figure 1: The Council for the Development of Cambodia's logo

The internship took place at The **Council for the Development of Cambodia (CDC)**, a senior executive agency of the Royal Government of Cambodia. Established in 1994 under the Law on Foreign Investment in the Kingdom of Cambodia, the CDC was created to serve as the central authority for investment promotion and economic development in the country. It operates as both the "Etat-Major" — the strategic command body — and the "One-Stop Service" for investors seeking to establish operations in Cambodia. The CDC's mandate spans the oversight of public and private investment, the rehabilitation and development of national infrastructure, and the administration of Special Economic Zones (SEZs) across the country.

1.2.2. Service of the Organization

The CDC delivers a range of critical services that underpin Cambodia's economic framework:

- **Investment Project Management:** The CDC evaluates, approves, and monitors Qualified Investment Projects (QIPs), ensuring that proposals align with national development priorities and comply with Cambodian investment law.
- **Strategic Development Coordination:** The agency manages Official Development Assistance (ODA) and coordinates partnerships with international donors and development institutions to finance infrastructure and capacity-building programs.
- **Special Economic Zone Administration:** The CDC oversees the legal, operational, and regulatory frameworks of Cambodia's SEZs, providing industrial investors with a structured and incentivized operating environment.

- **Investor Aftercare:** Beyond initial project approval, the CDC provides sustained support to both foreign and domestic investors to ensure operational continuity and long-term retention.

1.2.3. Vision and Mission

The CDC's vision is to establish Cambodia as a premier destination for high-value, sustainable investment in Southeast Asia. Its mission centers on reducing the "cost of doing business" — eliminating unnecessary bureaucratic friction, offering competitive fiscal incentives, and ensuring that Cambodia remains an accessible and attractive market for international capital.

Central to achieving this mission is the CDC's commitment to Digital Transformation. By adopting "GovTech" solutions across its operations, the organization aims to increase institutional transparency, accelerate response times, and modernize the way it communicates with investors. This internship project is a direct expression of this commitment: a purpose-built communication platform that enables CDC staff to manage investor correspondence with the speed and precision demanded by global business standards.

The CDC's strategic objectives are wide-ranging. They include attracting targeted foreign direct investment in sectors aligned with Cambodia's development roadmap, strengthening digital literacy and IT capacity within the public workforce, and centralizing development planning to ensure coherence between individual projects and national economic strategy. The CDC also works to streamline investment registration and approval procedures, reducing time-to-market for investors, and collaborates with the Cambodian Rehabilitation and Development Board (CRDB) to coordinate ODA flows effectively. In aggregate, these efforts position Cambodia as an increasingly competitive and diversified economy on the global stage.

1.2.4. Organizational Chart

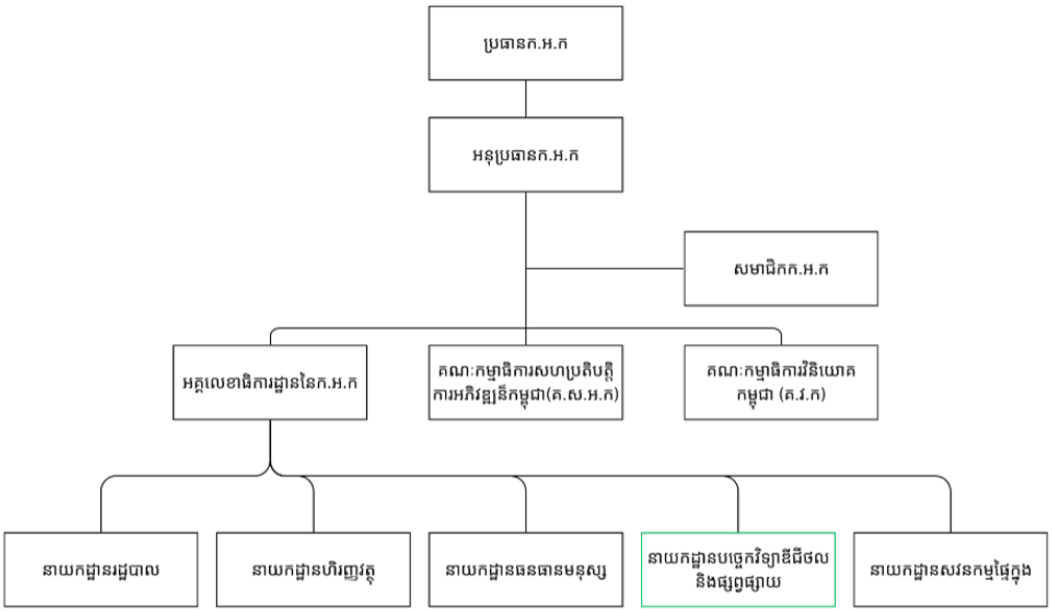

Figure 2: The CDC's organization chart

Figure 2 illustrates the institutional hierarchy of the Council for the Development of Cambodia. The organization is led by the President of the CDC, supported by the Vice-President. Key operating bodies include the Cambodia Investment Board (CIB), which handles private sector investment; the Cambodia Special Economic Zone Board (CSEZB), which governs SEZ policy; and the General Directorate, which oversees day-to-day administrative and technical functions.

My internship was hosted within the Information Technology and Public Relation Directorate (ITPR), a specialized unit situated under the General Directorate. The ITPR is responsible for the organization's digital infrastructure, internal systems, cybersecurity posture, and public communication channels. The internship project was initiated under the ITPR's broader mandate to equip CDC staff with modern, mobile-first tools that improve the efficiency and reliability of their day-to-day operations.

1.2.5. Address and Contact

The Council for the Development of Cambodia is located at:

- Address: Government Palace, Sisowath Quay, Wat Phnom, Phnom Penh, Cambodia.
- Phone: (+855) 23 981 154 / (+855) 23 427 597
- Website: www.cdc.gov.kh
- Email: info@cdc.gov.kh

II. PROJECT DEFINITION AND SCOPE

cdcIRM (CDC Investor Relations Management) is a specialized communication platform designed to serve as a digital bridge between the Council for the Development of Cambodia and global investors. While traditional email clients are built for general-purpose correspondence, this project is engineered for "High-Velocity Investor Relations," providing a focused environment for government-to-business (G2B) interactions.

2.1. Problem Statement and Conceptual Rationale

Currently, official correspondence is often fragmented across various platforms, leading to "context-switching" fatigue and potential delays in response times. cdcIRM addresses this by aggregating disparate email services into a single, unified, and intelligent interface. It moves beyond traditional email by integrating productivity tools such as OCR-based contact scanning and automated follow-up logic, ensuring the CDC maintains the precision expected by modern global investors.

The application focuses on three core pillars:

- **Immediacy:** Reducing the time spent on data entry and navigation.
- **Intelligence:** Assisting staff in maintaining relationships through automated reminders.
- **Integration:** Connecting directly with CDC's backend infrastructure while maintaining government-grade security protocols.

2.2. Targeted Impact

By centralizing communication and contact management, cdcIRM aims to revolutionize the CDC's internal workflows. This project supports the organization's objective of promoting transparency and national competitiveness on the global stage, ensuring that Cambodia remains an attractive hub for sustainable development.

2.3. Technical Architecture

The project follows a strict client-server architecture, with the mobile application and backend service developed as two independent, loosely coupled codebases that communicate exclusively through a well-defined REST API. This separation allows each layer to be maintained, tested, and deployed independently, a critical consideration in a government IT environment where change management procedures require clear boundaries between system components.

2.3.1. Backend Service

The backend is built on **NestJS**, organized as a monorepo containing three distinct packages: an **API** server that handles incoming HTTP requests and enforces business logic, a **Worker** process responsible for long-running background operations such as IMAP synchronization and push notification dispatch, and a **Shared** library that houses common type definitions, validation schemas, and utility functions consumed by both the API and the Worker. This architecture ensures that background tasks, particularly the continuous polling and listening on external mail servers, never compete with API request handling for server resources.

Authentication is implemented using **JSON Web Tokens (JWT)** with a dual-token strategy: a short-lived access token and a longer-lived refresh token. Token revocation is supported, enabling the system to forcefully terminate sessions when necessary, for example, when a device is reported lost or when a security audit demands it. Beyond simple authentication, the system implements **session trust levels**, a mechanism that assigns a confidence score to each active session based on metadata such as the device fingerprint, IP address, login time, and recent activity patterns. Sessions that fall below a configurable trust threshold can be flagged for re-authentication, adding a layer of adaptive security without burdening users with constant credential prompts.

Data persistence is managed through a relational database, with core models designed to represent mail accounts, email messages, attachments, contacts (person and company entities), access grants, and user sessions. Attachment storage is offloaded to a **CDN microservice**, ensuring that large file transfers do not degrade API performance and that files are served with appropriate caching headers for fast re-access.

2.3.2. Mobile Application

The mobile client is developed using **Flutter**, primarily targeting iOS and Android from a single codebase. Additionally, the application is compiled as a Web application specifically optimized for deployment as a **Telegram Mini App**, extending the platform's reach to officials who prefer browser-based or in-app messaging workflows.

Navigation is managed through **GoRouter**, with route-level guards that enforce authentication boundaries, unauthenticated users are restricted to onboarding and authentication screens, while authenticated users are routed directly into the application shell.

The application's state management architecture relies on a layered provider model: global providers handle cross-cutting concerns such as authentication state, API client configuration, secure token storage, and routing; feature-level providers manage the state of individual

screens such as the inbox, composer, and contact detail views. Data models are generated using **freezed** with **json_serializable**, providing immutable value objects and automatic JSON serialization, which eliminates an entire class of runtime parsing errors.

A dedicated theming system supports both dark and light modes natively. Crucially, for the Telegram Mini App deployment, a specialized theme engine is implemented. This engine consumes the raw color parameters (such as background color, text color, and accent hints) passed by the Telegram WebApp API and programmatically constructs a valid, highly readable **Material Theme palette**. This ensures that when the application is launched within Telegram, it seamlessly inherits the user's personal Telegram appearance settings while strictly adhering to Material Design contrast and accessibility guidelines. The application also features deep integration with Telegram's WebApp APIs, enabling native-like interactions such as back button, haptic feedback, dynamic theme change listeners, and seamless viewport expansion.

The application is fully localized in **Khmer (KM)** and **English (EN)**, with all user-facing strings externalized and resolved at runtime based on the user's language preference.

2.4. Functional Scope

The functional scope of cdcIRM is organized into four capability domains, each corresponding to a phase of the development timeline.

2.4.1. Identity and Access Management

This domain encompasses the complete authentication lifecycle. New users are onboarded through a structured onboarding flow that communicates the application's purpose and permissions before presenting registration and login screens. Upon authentication, the client securely stores access and refresh tokens in platform-appropriate secure storage and configures an HTTP interceptor that automatically attaches tokens to outbound requests and handles token refresh when the access token expires. On the backend, each session is tagged with trust-level metadata, and the revocation endpoint allows the system to invalidate sessions selectively.

2.4.2. Email Management

The email management domain is the core of the application. On the backend, a dedicated Worker process connects to external mail servers via the **IMAP protocol**. The initial implementation specifically targets the CDC's **cPanel-based** mail infrastructure. However, the synchronization layer is designed with a modular provider architecture, abstracting provider-specific behaviors behind a unified interface, allowing new mail servers or providers to be integrated in the future without refactoring the core mail processing logic.

New mail events fetched by the Worker are propagated to the client through a push notification pipeline built on **Firebase Cloud Messaging (FCM)**, ensuring that officials are alerted in real time, both in the foreground and when the application is in the background, with tap-to-route logic that opens the relevant email directly.

The API exposes full CRUD operations for emails: listing with server-side pagination, retrieving individual messages with HTML body rendering, composing and sending, deleting, and moving between folders. Full-text search is supported, with filters for account, date range, sender, and folder. The mobile client presents this through a multi-tab inbox shell, each tab corresponds to a configured mail account, with an additional tab for department-level shared mailboxes. The email detail screen renders HTML content within a thread view, and the compose screen integrates a rich text editor with to, cc, and bcc recipient fields. Reply and forward operations pre-populate the composer with the appropriate context. Attachments are handled through a dedicated picker and preview interface, with uploads routed to the CDN microservice and downloads streamed directly to the device.

2.4.3. Collaborative Access and Account Sharing

A distinctive feature of cdcIRM is its support for **departmental mail account sharing**. Government investor relations often require that multiple staff members access a shared inbox, for example, a general inquiries mailbox monitored by a rotating team. The backend implements an access grant system that allows administrators to assign read, write, or administrative permissions on a per-account, per-department basis. A special `BLOCKED` sentinel status is used to instantly revoke access without deleting the grant record, preserving an audit trail. The mobile client surfaces shared accounts alongside personal accounts in the inbox shell, and the permission resolution logic is verified end-to-end during a dedicated permission audit phase.

2.4.4. Contact and Entity Management

The contact management domain treats contacts as structured entities, specifically **persons** and **companies**, rather than simple address book entries. Each entity carries typed fields (name, role, organization, phone, email, address) and can be linked to any number of email messages through an indexing system, enabling staff to view the complete correspondence history associated with a given contact without manual searching.

The system also includes a **card scan** capability: the mobile client opens the device camera to capture a business card image, sends it to a backend endpoint that proxies the image to an external OCR service, and maps the recognized fields to a new or existing contact entity. The user is then presented with a confirmation screen to review, correct, and save the extracted

data. Additionally, any email address appearing in a mail message can be promoted to a contact entity through a **save-as-contact** flow, reducing manual data entry during high-volume correspondence periods.

2.5. Non-Functional Requirements

2.5.1. Security

Government-grade security is a non-negotiable constraint. All API communication occurs over HTTPS. Authentication tokens are stored in platform secure storage (Keychain on iOS, EncryptedSharedPreferences on Android, or secure Web storage for the Telegram Mini App). The session trust-level system provides behavioral anomaly detection without requiring a full multi-factor authentication infrastructure. The backend enforces rate limiting and strict input validation to mitigate abuse, and all endpoints return structured error codes that the client can map to appropriate user-facing messages without exposing internal system details.

2.5.2. Reliability and Performance

The separation of the API and Worker processes ensures that background mail synchronization never degrades the responsiveness of interactive operations. Attachments are served through a CDN, reducing load on the API server. Pagination is enforced on all list endpoints to prevent unbounded data transfer. The mobile client implements offline-aware empty states and error banners that communicate network issues to the user without crashing or silently failing.

2.5.3. Localization, Accessibility, and Platform Adaptation

The application supports Khmer and English throughout the entire interface, including dynamically rendered email metadata, error messages, and settings labels. The standard theming system ensures readability in both light and dark modes, with color tokens defined at the design-system level to prevent inconsistencies. For the Telegram Mini App variant, the specialized theme engine guarantees that platform-adaptive colors do not compromise text legibility or interactive element contrast, maintaining accessibility standards regardless of the user's Telegram theme configuration.

2.5.4. Testability and Delivery

The backend development includes a dedicated E2E and integration testing phase that verifies critical flows, authentication, mail fetching, permission resolution, and contact creation, against a real database instance. Deployment is managed through a CI pipeline with environment-specific configuration and automated database migrations. The final mobile deliverable is a **demo build** with release-optimized configuration, branded application icon, and splash

screen, prepared for internal distribution within the CDC across native stores and the Telegram platform.

2.6. System Boundaries and Integration Points

cdcIRM does not operate in isolation. It integrates with three primary external systems: the CDC's **cPanel IMAP servers** (for mail fetching and listening, consumed via a modular provider interface), **Firebase Cloud Messaging** (for cross-platform push notification delivery to mobile and web targets), and an **OCR service** (for business card scanning). Furthermore, the web-deployed variant of the application integrates deeply with the **Telegram WebApp environment**, consuming Telegram's native APIs for dynamic UI theming, haptic feedback, and viewport management.

The application does not replace the CDC's existing cPanel mail infrastructure; rather, it sits atop it as a specialized consumer, aggregating and enriching the data for a specific use case. This boundary is deliberate: it ensures that cdcIRM can be introduced incrementally, whether via native mobile or Telegram, without disrupting existing email workflows for staff who are not yet onboarded.

2.7. Methodology

III. LITERATURE REVIEW

To ensure the architectural decisions made during the development of cdcIRM were both industry-aligned and academically sound, a comprehensive review of existing systems and underlying computer science theories was conducted. This section establishes the benchmark against which cdcIRM is measured and justifies the engineering patterns chosen to overcome the limitations of off-the-shelf solutions.

3.1. Existing Systems Analysis

The landscape of email communication has shifted from simple message exchange to complex productivity ecosystems. While traditional clients focus on broad feature sets, modern "power-user" clients prioritize speed and organizational efficiency. However, none of these existing solutions adequately address the specific constraints of Cambodian government infrastructure.

3.1.1. Enterprise Industry Standards: Gmail & Outlook

- **Gmail:** Recognized for its powerful search capabilities and deep integration with Google Workspace, having pioneered features like "Smart Compose" and tabbed categorization. However, for high-stakes investor relations, users often find its interface cluttered and prone to distractions from non-essential automated mail. [1]
- **Microsoft Outlook:** The definitive standard for "locked-down" institutional organizations. Its strength lies in robust calendar integration and enterprise-grade security (S/MIME). While the CDC utilizes standard mail protocols, Outlook's mobile interface is frequently criticized for being heavy and unintuitive for rapid, on-the-go tasks. More importantly, deploying Outlook does not solve the need for localized, Khmer-language investor context or native integration with platforms like Telegram. [2]

3.1.2. High-Productivity & AI-Driven Clients: Superhuman & Spark

- **Superhuman:** Widely considered the fastest email experience currently available. It utilizes a "keyboard-first" philosophy and a "Split Inbox" to separate human-sent emails from automated notifications. A key relevant feature is AI-powered follow-ups, which nudge users to maintain communication momentum—a critical need for investor relations. cdcIRM adapts this "Split Inbox" concept physically into its structure (personal vs. department shared mailboxes) tailored for mobile use. [3]
- **Spark Mail:** Emphasizes team collaboration and "Smart Categorization." Its standout feature allows users to screen new senders, a valuable concept for government

officials who must filter high volumes of unsolicited investor inquiries. cdcIRM integrates a similar philosophy at the contact management level, allowing officials to categorize entities (persons vs. companies) and track interaction history. [4]

3.1.3. Specialized CRM & Contact Integration

Platforms such as **Salesforce Mobile** and **HubSpot** facilitate lead management by allowing users to scan business cards to create CRM entries. While these provided the inspiration for the cdcIRM contact scanning module, they are proprietary, high-cost SaaS platforms. They do not natively integrate with private cPanel IMAP servers or support the Khmer language natively. cdcIRM bridges this gap by building an OCR-powered contact pipeline directly into the mail client, eliminating the need for a separate, expensive CRM license.

3.2. Engineering Theory and Design Patterns

To bridge the gaps identified in existing systems and adhere to the constraints of the CDC's infrastructure, this project utilizes several foundational computer science patterns.

3.2.1. The Adapter Pattern for Mail Provider Abstraction

A primary challenge in email aggregation is "Interface Incompatibility" between disparate providers. Academic literature identifies the **Adapter Design Pattern** as a structural solution that allows incompatible interfaces to collaborate. [5] In cdcIRM, the backend Worker initially targets the CDC's cPanel-based IMAP servers. By implementing a unified `MailAdapter` interface in the NestJS shared library, the core synchronization logic remains completely decoupled from cPanel-specific IMAP commands. This ensures that if the CDC migrates to Microsoft Exchange, Google Workspace, or CDC's own mail server in the future, a new adapter can be written without modifying a single line of the core mail processing or push notification code.

3.2.2. Zero Trust and Context-Aware Security

Traditional security models often rely on a "Binary Perimeter" (authenticated vs. unauthenticated). Modern enterprise security has shifted toward **Zero Trust Architecture (ZTA)**, which posits that trust must be continuously verified. [6] The cdcIRM "Trust Engine" applies these principles by calculating a dynamic **Trust Score** for every active session. Rather than relying solely on a valid JWT, the system evaluates environmental metadata—such as IP geolocation shifts, device fingerprint changes, and time-of-day anomalies. If a session's trust level degrades below a configurable threshold, the system forces re-authentication, providing

government-grade security without burdening officials with constant Multi-Factor Authentication (MFA) prompts.

3.2.3. Adaptive Color Theory and Cross-Platform UI Consistency

Deploying a Flutter application natively on iOS/Android and as a Web-based Telegram Mini App presents a unique theming challenge. Telegram allows users to apply highly customized, user-generated themes (dark, light, custom accent colors) and passes these raw RGB values to the Mini App via its WebApp API. Standard practices often result in broken contrast or unreadable text when blindly applying external hex codes to a Material Design framework. cdcIRM employs an algorithm rooted in **Adaptive Color Theory** to solve this. A specialized theme engine takes the raw Telegram color variables and programmatically maps them to a valid **Material Theme Palette** (calculating appropriate `onPrimary`, `surface`, and `onSurface` variants). This ensures that regardless of how unconventional a user's Telegram theme is, the cdcIRM interface strictly adheres to WCAG accessibility contrast ratios while perfectly matching the user's aesthetic expectations.

3.2.4. Event-Driven Architecture and Asynchronous Processing

Handling large volumes of investor correspondence without degrading API performance requires strict separation of concerns. Instead of relying on expensive, persistent HTTP connections like WebSockets—which are difficult to maintain natively across Flutter Mobile and Telegram Web simultaneously—cdcIRM utilizes an **Event-Driven Architecture**. The NestJS Worker process acts as an isolated event consumer, listening for IMAP IDLE pushes. When a new mail event is detected, the Worker processes the payload and triggers a **Firebase Cloud Messaging (FCM)** broadcast. This pattern ensures that push notification delivery is handled entirely by Google's infrastructure, allowing the cdcIRM API server to remain stateless and highly responsive.

3.2.5. Cursor-Based Pagination for High-Volume Data

For the inbox list views, traditional "Offset" pagination (e.g., `?page=2&limit=20`) proves inadequate in an active mail environment. If a new email arrives while a user is scrolling, offset pagination can cause records to skip or duplicate on the screen. [7] cdcIRM implements **Cursor-based Pagination** on all email list endpoints. By querying against a unique, indexed cursor (such as a composite of the message timestamp and ID), the system guarantees a stable, high-performance "infinite scroll" experience. Even if thousands of new emails are synced by the background Worker, the user's current scrolling context remains perfectly stable and accurate.

IV. PROJECT ANALYSIS

The system analysis phase involved translating the CDC's institutional goals into a set of technical mandates. This section outlines the functional and non-functional requirements that governed the development of the cdcIRM platform.

4.1. Requirements Specification

4.1.1. Functional Requirements

Functional requirements define the specific actions and behaviors the application must perform to satisfy user needs:

Table 1: Functional requirements of the cdcIRM system

ID	Feature	Description
1	Email Linking	Allow the user to link their mailboxes from various email providers
2	Inbox Listing	The system shall display a list of email messages with essential metadata such as sender, subject, timestamp, and status (read/unread).
3	Investor Contact Scanning	The system shall capture an image from the device camera and extract contact data such as name, company, phone number, and email address.
4	Investor Contact Review and Edit	The system shall allow users to review and correct OCR results before saving them.
5	Investor Contact Storage	The system shall save validated contact information into the system for future communication use.
6	AI Assistance	The system shall support AI-based drafting or response suggestions for selected email messages.
7	Notification Handling	The system shall notify users about pending follow-ups, important messages, or system events.
8	Language Preference	The system shall support application localization and allow switching between Khmer and English.

4.1.2. Non-Functional Requirements

- **Multi-lingual Support:** The system must allow users to toggle between Khmer and English instantly, ensuring all system messages, data categories, and UI labels are localized.
- **Security (Contextual Trust):** The system must implement token-based authentication (JWT) and a "Trust Engine" that evaluates session metadata (IP, location, and device ID) to protect sensitive government data.
- **Performance (Cursor-Based Pagination):** To handle high-velocity email feeds without latency, the system must utilize cursor-based pagination to ensure a stable and fluid scrolling experience.
- **Scalability (Adapter Architecture):** The backend must be provider-agnostic, utilizing the Adapter Design Pattern to allow the integration of new email services without structural code changes.
- **Usability:** The interface must adhere to the provided design documentation provided by the company to ensure consistent design and facilitate the usability of CDC staffs.

Table 2: Non-functional requirements

Feature	Description
Usability	The user interface should be simple, readable, and optimized for mobile use by non-technical and technical staff alike. Important actions should be reachable with minimal taps.
Security	The system should use token-based authentication (JWT), secure API communication over HTTPS, and protected local storage for sensitive session data.
Reliability	The system should handle network failures gracefully by providing clear feedback and retry options without crashing.
Maintainability	The codebase should follow a modular architecture with clear separation between data models, services, state management, and UI components.
Scalability	The design should support future additions such as calendar integration, investor tagging, analytics, and advanced AI modules.
Compatibility	The mobile application should run on modern Android and iOS devices used by CDC staff.
Localization	The system should support Khmer and English text resources and be designed to allow adding more languages in the future.

4.2. Use Case Analysis

To understand the interaction between CDC officials and the cdcIRM ecosystem, the following primary use cases were identified:

4.2.1. Account Linking

Figure 3: Account Linking Use Case Diagram

Figure 3 Outlines CDC official authenticates via the CDC internal portal and links their professional Gmail or Outlook account via a secure OAuth2 handshake.

4.2.2. Scanning Business Cards

Figure 4: Business Card Scanning Use Case Diagram

Figure 4 During an investment forum, an official scans an investor's business card using the OCR module, which automatically populates the CDC's centralized investor database.

4.2.3. Primary Actor

The primary actor of the system is:

- **CDC Staff User:** Authorized officer using the mobile application for investor communication and follow-up management.

4.2.4. Supporting Actors / External Systems

- **CDC Backend API:** Provides authentication, user profile, email synchronization, and contact-related services.
- **OCR Engine:** Extracts text from captured business card or ID images.
- **Notification Service:** Delivers local or push notifications for reminders and updates.
- **AI Service:** Generates drafting suggestions and follow-up assistance.

4.2.5. Use Case Diagram (Conceptual)

The formal use case diagram can be inserted as a figure in the final report. For documentation clarity, the actor-to-use-case relationships are listed below.

- **CDC Staff User** → Login, View Inbox, Read Email, Compose Email
- **CDC Staff User** → Scan Contact, Save Contact, Create Follow-up, Receive Reminder
- **CDC Staff User** → AI Draft Suggestion
- **CDC Backend API** ⟷ Authentication, Email Sync, Contact Storage
- **OCR Engine** ⟷ Scan Contact
- **Notification Service** ⟷ Receive Reminder
- **AI Service** ⟷ AI Draft Suggestion

Figure 5: Use case diagram of cdcIRM (Placeholder)

4.3. Process Flow and Activity Concepts

This section describes the main activity flow of the application from login to communication and follow-up.

4.3.1. Core Workflow (Narrative)

1. The user opens the application and signs in using CDC credentials.
2. The system validates credentials through the backend and stores a secure session token.
3. The inbox is loaded and synchronized with the backend email service.
4. The user may read emails, compose new messages, or scan a contact card.
5. If scanning is used, OCR extracts text and returns candidate contact fields.
6. The user reviews and edits extracted data before saving.
7. The user can create a follow-up reminder for an investor conversation.
8. The system schedules a reminder and notifies the user at the specified time.

4.3.2. Activity Diagram (Conceptual)

Figure 6: Activity Diagram for Main User Workflow (Placeholder)

4.4. Data and Database Design

Although the internship project focuses on mobile application development, a clear data model is required to ensure smooth integration with backend APIs and future expansion. The database design presented here represents the conceptual structure of the main entities used by the system.

4.4.1. Main Entities

The system revolves around the following core entities:

- **User:** CDC staff account and profile information
- **Message:** Email metadata and content
- **Entity:** Investor or company details
- **Attachment (optional):** File metadata associated with emails
- **Audit Fields:** Created/updated timestamps for traceability

4.4.2. Conceptual Entity Relationships

- One **User** can access many **Mailboxes**
- One **User** can create many **Follow-up Reminders**
- One **Entity** can be linked to many **Email Messages**
- One **Email Message** can contain multiple **Attachments**
- One **Follow-up Reminder** may reference a **Entity** and/or an **Email Message**

Figure 7: Database Table (Placeholder)

4.4.3. Database Design Considerations

The following design principles are important for this project:

- **Normalization:** Separate contacts, emails, and reminders to avoid duplicate data.
- **Traceability:** Include timestamps for creation and updates.
- **Extensibility:** Keep optional fields flexible for future modules (e.g., investor tags, categories, priority).
- **Security:** Avoid storing sensitive tokens or secrets directly in business tables.

- **Integration-first:** The mobile app may not directly access the database, but entity design should align with backend API responses.

4.4.4. Sample Data Dictionary (Selected Fields)

Table 3: Selected Data Dictionary

Table	Field	Type	Description
user	id	UUID / String	Unique identifier of CDC staff user.
message	thread_id	String	Groups related messages into one conversation thread.
message	is_read	Boolean	Read/unread state for UI display.

4.5. System Architecture

The project follows a Decoupled Architecture to ensure maintainability and scalability.

4.6. Project Timeline

Table 4: Activities timeline

Tasks	Weeks																							
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Requirement Analysis																								
Setup Project																								
Development Phase																								
Testing																								
Bug Fixes and UAT																								
Documentation																								

(DRAFT) Table 4 Outlines the timeline for the entire Internship 2 project span. For the first month, I spent most of the time initializing the project—including the file structure, dependencies and others.

4.7. Analysis of Technical Architecture

To support maintainability and future growth, the mobile application adopts a layered architecture.

4.7.1. Proposed Application Layers

- **Presentation Layer:** Flutter UI screens, widgets, and user interactions
- **State Management Layer:** Handles screen state, loading state, and user actions
- **Domain/Logic Layer:** Business rules such as validation, reminder logic, and work-flow handling

- **Data Layer:** API clients, local storage, serialization, and repository implementations
- **External Services Layer:** OCR, AI assistance, notification service, and backend APIs

This separation improves:

- Code readability
- Testing capability
- Feature scalability
- Team collaboration during future maintenance

4.7.2. API and Data Exchange Considerations

The backend communication is expected to use JSON-based REST APIs. During analysis, the following integration concerns were considered:

- Authentication token refresh and secure storage
- Standardized API response format (success/error)
- Consistent date-time parsing (ISO 8601)
- Pagination for inbox and contact lists
- Snake_case API keys mapped to camelCase mobile models
- Error handling for unstable network conditions

These considerations influenced the model and service design in the implementation phase.

V. DETAIL CONCEPT

This chapter presents the detailed concept of the proposed **cdcIRM** system. While the previous chapter focused on analysis and requirements, this chapter translates those findings into a concrete design structure, including the logical architecture, physical architecture, and the technologies and tools required for implementation.

The purpose of this chapter is to define how the system is organized technically before implementation, so that development remains consistent, maintainable, and aligned with CDC operational needs.

5.1. Logical Architecture

The logical architecture describes the software structure of the **cdcIRM** mobile application and how its internal components interact. The system follows a layered and modular design to separate responsibilities and simplify maintenance.

5.1.1. Architecture Style

The proposed architecture uses a **layered application design** with modular features. Each layer has a specific role:

- **Presentation Layer** handles UI screens, widgets, and user interactions.
- **State Management Layer** manages UI state, asynchronous loading, and actions.
- **Domain Layer** contains business rules and workflows.
- **Data Layer** handles API communication, local storage, and model serialization.
- **Service Integration Layer** connects to OCR, notification, and optional AI services.

This design improves readability, testing, and future expansion.

5.1.2. Logical Components

Table 5: Logical components of the cdcIRM application

Component	Responsibility
Authentication Module	Handles login, token persistence, token refresh, logout, and access control for protected routes.
User Profile Module	Retrieves and stores authenticated user profile data such as name, role, email, and language preference.
Inbox Module	Displays email list, supports pagination, search, read/unread state, and navigation to message details.
Email Compose Module	Supports composing, replying, and forwarding messages with form validation and API submission.
Contact Scan Module	Captures image input and processes OCR results into structured contact fields.
Contact Management Module	Allows review, edit, save, and retrieve investor contact records.
Follow-up Module	Creates and manages reminders linked to contacts or email threads, including due status.
Notification Module	Schedules and displays reminder notifications for follow-up activities.
Localization Module	Provides Khmer/English language support and handles text resource loading.
Shared Core Module	Contains reusable utilities, API response wrappers, error handling, constants, and local storage services.

5.1.3. Layer Interaction Flow

The following sequence summarizes how layers interact during a common user action, such as logging in or opening the inbox:

1. The user triggers an action from the UI.
2. The presentation layer forwards the action to state management.
3. State management calls domain logic or a repository.
4. The repository accesses remote APIs or local storage.
5. The data result is mapped into app models.
6. State is updated and reflected back in the UI.

This pattern ensures that user interface code remains independent from backend communication logic.

5.1.4. Data Flow Concept

The application uses JSON-based API communication and local persistence for session-related data. Data flows through the system as follows:

- **Remote data:** backend API responses (JSON)
- **Model conversion:** snake_case JSON fields mapped to camelCase app models
- **State update:** parsed models stored in memory/state controllers
- **Local persistence:** tokens, preferences, and selected cached values stored locally
- **UI rendering:** widgets consume prepared state models

Date/time values should be converted from ISO 8601 strings into native date-time objects in the model layer to ensure consistent formatting and sorting.

5.2. Physical Architecture

The physical architecture describes the deployment environment and the hardware/ software elements that host and connect the **cdcIRM** system.

The project is primarily a **mobile client application** connected to CDC-managed back-end services over a secure network. The mobile app does not directly connect to the database; instead, it communicates through APIs.

5.2.1. Deployment Overview

The physical architecture contains the following main nodes:

- **Mobile Device (Client):** Android or iOS device used by CDC staff
- **CDC Backend API Server:** Handles authentication, email operations, contacts, reminders, and tores user, contact, email metadata, and follow-up records
- **File/Attachment Storage (optional):** Stores or proxies attachment files
- **OCR Service:** Processes images for contact extraction
- **Notification Service:** Handles push or local reminder notifications
- **AI Service:** Provides smart drafting assistance

Figure 8: Physical architecture of the cdcIRM system (Placeholder)

5.2.2. Physical Communication Flow

The high-level communication flow is:

1. User opens the mobile app and submits credentials.
2. Mobile app sends authentication request to the CDC backend API over HTTPS.
3. Backend validates credentials and returns access/refresh tokens.
4. Mobile app requests inbox, profile, contact, and follow-up data through APIs.
5. Backend reads/writes data to the database and returns JSON responses.
6. OCR and AI-related features call their respective services via backend.
7. Notification service triggers reminder alerts on the device.

5.2.3. Client Device Requirements

The mobile client should run on modern smartphones commonly used by CDC staff. Minimum practical conditions include:

- Android or iOS device with stable internet connection
- Camera support for OCR/business card scanning
- Notification permission enabled for reminder features

5.2.4. Security Considerations in Physical Design

Because the application handles official communication and contact data, physical deployment should consider the following security measures:

- HTTPS-only API communication
- Permission-based access control
- JWT-based session authentication

- Secure local storage for tokens
- Input validation and server-side authorization checks
- Controlled access to OCR/AI services
- Logging and auditability on backend services

These controls reduce the risk of unauthorized access and data leakage.

5.3. Tools and Technology Requirements

This section defines the technologies and development tools used to design, implement, test, and maintain the **cdcIRM** internship project.

The selected technologies were chosen based on:

- compatibility with mobile development goals
- maintainability for future CDC developers
- performance and developer productivity
- support for localization and modular architecture

5.3.1. Technologies

a. NestJS (TypeScript)

Figure 9: NestJS logo

NestJS is the primary backend framework used to build the **cdcIRM** server-side architecture.[1] It was selected for its modular structure, which perfectly aligns with the project's monorepo requirement—separating the REST API, the background IMAP Worker, and shared utility libraries into distinct, maintainable packages. Its native dependency injection system simplifies the management of complex services like authentication, database connections, and mail provider adapters.

[1] https://nestjs.com

b. Flutter (Dart)

Figure 10: Flutter logo

Flutter is the main framework used to develop the **cdcIRM** mobile application. It supports cross-platform development, allowing the same codebase to run on both Android and iOS devices with consistent UI and behavior.[2] Crucially, its web compilation capabilities allow the application to be deployed as a high-performance **Telegram Mini App**, bridging native mobile and web-based workflows within a single unified codebase.

c. Firebase Cloud Messaging (FCM)

Figure 11: Firebase logo

Firebase Cloud Messaging is the push notification infrastructure used to alert officials of new incoming mail in real time.[3] Integrated deeply into both the NestJS Worker process and the Flutter client, FCM handles the reliable delivery of background alerts and foreground data messages. The client utilizes FCM routing logic to intercept notification taps and navigate the user directly to the relevant email detail screen.

d. IMAP Protocol (cPanel)

Figure 12: cPanel logo

The Internet Message Access Protocol (IMAP) is the underlying standard used by the backend Worker to communicate with the CDC's mail servers. The initial implementation is specifically tailored to connect to the CDC's cPanel-based email infrastructure.[4] By consuming

[2] https://flutter.dev
[3] https://firebase.google.com
[4] https://www.cpanel.net

IMAP IDLE commands, the Worker listens for new mail events in real time, triggering downstream processing and push notifications without resorting to inefficient constant polling.

e. Telegram WebApp API

Figure 13: Telegram logo

The Telegram WebApp API is a specialized integration layer used to transform the Flutter web build into a native-feeling Telegram Mini App. It provides deep platform access, allowing the application to read the user's localized Telegram theme parameters to construct a valid Material color palette dynamically. It also enables native-like device features such as haptic feedback, theme change listeners, and safe area viewport expansion.

f. GoRouter (Dart)

GoRouter is the declarative routing solution implemented in the Flutter application. It manages all screen transitions and enforces authentication boundaries through route-level guards. This ensures that unauthenticated users are strictly confined to the onboarding and login flows, while authenticated users are seamlessly routed into the main application shell based on their session state.

g. Freezed & json_serializable (Dart)

Freezed, paired with json_serializable, is the code-generation stack used for all data modeling in the mobile application. It generates immutable value objects and robust JSON serialization/deserialization logic for complex backend responses (like email threads and contact entities). This eliminates an entire class of runtime parsing errors and ensures type safety when passing data across different providers and screens.

h. JSON Web Tokens (JWT)

JWT is the stateless authentication standard securing all API communications between the Flutter client and the NestJS server. The project implements a strict dual-token strategy—using short-lived access tokens for request authorization and long-lived refresh tokens to silently obtain new access tokens. The backend maintains token revocation capabilities to support the session trust level system, allowing immediate session termination if anomalous activity is detected.

These Dart packages are used to define API models with reduced boilerplate and safe JSON serialization. They improve code maintainability and ensure consistent mapping between snake_case API fields and camelCase Dart properties.

i. Local Storage / Secure Storage

Storage Logo Placeholder

Local storage is used to persist user preferences and session-related values on the device. Sensitive data such as authentication tokens should be stored using secure storage mechanisms to improve application security.

j. OCR Technology

OCR Logo Placeholder

OCR technology is used to extract contact information from business cards or ID images captured with the mobile camera. This reduces manual data entry time and improves workflow efficiency for staff handling investor contacts.

k. Notification Service

Notification Logo Placeholder

The notification service is used to deliver follow-up reminders and important alerts to users. It supports better task tracking by helping staff respond to investor communications on time.

l. Flutter Localization (l10n / ARB)

Localization Logo Placeholder

Flutter localization tools are used to support multilingual content in the app, especially Khmer and English. This ensures the interface remains accessible and appropriate for different users within the CDC environment.

5.3.2. Tools

a. Visual Studio Code / Android Studio

IDE Logo Placeholder

These development environments are used for writing, debugging, and organizing the Flutter project source code. They provide features such as code completion, terminal integration, and plugin support for Flutter development.

b. Flutter SDK

Flutter SDK Logo Placeholder

The Flutter SDK provides the framework libraries, build tools, and command-line utilities required to run and compile the mobile application. It is the core toolchain used throughout the development process.

c. Git

Git Logo Placeholder

Git is used for version control to track code changes and manage development history. It helps maintain project stability and makes collaboration easier when multiple contributors work on the same codebase.

d. GitHub / GitLab

GitHub / GitLab Logo Placeholder

A repository hosting platform is used to store and manage the project remotely. It supports source backup, collaboration workflows, and issue tracking during the internship development period.

e. Postman / Insomnia

Postman / Insomnia Logo Placeholder

API testing tools are used to verify backend endpoints before integrating them into the mobile application. They help inspect request payloads, responses, and error handling behavior during development.

f. Android Emulator / iOS Simulator / Physical Device

Device Testing Logo Placeholder

Testing environments are required to validate the application on different platforms and screen sizes. Physical devices are especially important for camera-based features such as OCR and real notification behavior.

g. Figma

Figma Logo Placeholder

Figma is used to design and preview interface layouts before implementation. It helps organize screens, user flows, and visual consistency for the mobile application during planning and refinement.

h. Typst

Typst Logo Placeholder

Typst is used to prepare this very internship project report. It offered structured formatting, figures, and chapter organization. It provides a clean workflow for maintaining technical documentation with consistent layout.

i. draw.io / Lucidchart / PlantUML

Diagram Tool Logo Placeholder

Diagramming tools are used to create architecture, use case, and activity diagrams for the report. These visual materials improve clarity and help explain system structure and workflows more effectively.

5.3.3. Development Environment Requirements

A stable development environment should include:

- Flutter SDK properly configured
- Android SDK and emulator support
- Xcode (for iOS build/testing on macOS)
- Git installed and configured
- Access to CDC backend API test environment
- Access credentials for OCR/notification services (if enabled)

These requirements ensure the application can be built and tested reliably during the internship period.

VI. IMPLEMENTATION

…

6.1. Project Structures

…

6.2. Sequence Diagram

…

VII. CONCLUSION

…

7.1. Completed Functions

…

7.2. Incomplete Functions

…

7.3. Challenges

…

7.4. Experience

…

7.5. Perspective

…

7.6. Future Work

…

REFERENCES

[1] Google LLC, “Gmail.” [Online]. Available: https://mail.google.com/

[2] Microsoft Corporation, “Microsoft Outlook.” [Online]. Available: https://outlook.live.com/

[3] Superhuman Labs, Inc., “Superhuman.” [Online]. Available: https://superhuman.com/

[4] Readdle Inc., “Spark Mail.” [Online]. Available: https://sparkmailapp.com/

[5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston, MA, USA: Addison-Wesley Professional, 1994.

[6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero Trust Architecture,” technical report NIST SP 800-207, 2020. doi: 10.6028/NIST.SP.800-207.

[7] M. Winand, “Keyset Pagination.” 2013.

APPENDICES